AI Coding Study Analysis

Adam Kiehl

2026-04-16

Analysis Overview

This document imports and reshapes coding-study data, quantifies inter-coder agreement, models agreement and coding time, and produces diagnostic and presentation figures. Code statements from the source file are preserved; organizational prose, chunk labels, and comments have been added for maintainability.

1. Data Engineering

1.1 Import Data

Imports the coding records, reports their structure, removes unused columns, and retains complete coder observations.

```
# Read, inspect, reduce, and completeness-filter the primary record-level dataset.
# 1) Import the CSV
raw_data <- readr::read_csv("../data_entry/records_df_done.csv", show_col_types = FALSE)

# 2) Print a summary including column dtypes and total number of rows
cat("=== DataFrame Summary ===\n")
```

```
## === DataFrame Summary ===
```

```
cat("Total rows:", nrow(raw_data), "\n")
```

```
## Total rows: 500
```

```
cat("Total columns:", ncol(raw_data), "\n\n")
```

```
## Total columns: 29
```

```
# Column dtypes + preview (built-in tidyverse helper)
dplyr::glimpse(raw_data)
```

```
## Rows: 500
## Columns: 29
## $ record_id     <dbl> 228755, 18284, 100014, 258616, 46602, 26645, 109528, 354~
## $ diagnosis     <chr> "episcleritis lethargy", "apparently healthy", "grade ii~
## $ assessment    <chr> "carl, a 1.5 year old male castrated cavadoodle, was pre~
## $ pathology     <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
## $ tracy         <chr> "815008\n214264003", "102499006", "93886007\n94392001\n4~
## $ nadia         <chr> "815008\n214264003", "102499006", "414653009\n94392001\n~
## $ katie         <chr> "815008\n214264003", "102499006", "414653009\n94392001\n~
## $ terri         <chr> "815008\n214264003", "102499006", "372130007\n94392001\n~
## $ beth          <chr> "815008\n214264003", "102499006", "432525007\n414653009\~
## $ naija         <chr> "815008\n214264003", "102499006", "414653009\n94392001",~
## $ erik          <chr> "815008\n214264003", "102499006", "414653009\n126488004"~
```

```
## $ caitlin       <chr> "815008\n214264003", "102499006", "414653009\n94392001",~
## $ ai            <lgl> FALSE, FALSE, FALSE, FALSE, TRUE, TRUE, TRUE, FALSE, FAL~
## $ time_tracy    <dbl> 36.071526, 16.843228, 181.776396, 210.795757, 97.479476,~
## $ time_nadia    <dbl> 30.41589, 13.81851, 91.82989, 269.10510, 110.63021, 185.~
## $ time_katie    <dbl> 59.893194, 20.228365, 117.571965, 506.954308, 67.948004,~
## $ time_terri    <dbl> 37.708881, 15.773790, 145.624207, 242.443283, 101.606405~
## $ time_beth     <dbl> 32.89564, 18.79442, 217.20397, 377.58321, 59.84232, 296.~
## $ time_naija    <dbl> 32.295973, 15.067100, 50.129754, 129.346031, 37.544660, ~
## $ time_erik     <dbl> 72.019010, 27.284061, 71.617956, 110.278512, 50.735787, ~
## $ time_caitlin  <dbl> 30.319281, 15.814404, 55.846680, 146.831581, 39.448095, ~
## $ order_tracy   <dbl> 313, 337, 331, 450, 95, 171, 71, 395, 348, 133, 54, 394,~
## $ order_nadia   <dbl> 448, 484, 428, 361, 141, 158, 54, 421, 317, 68, 7, 469, ~
## $ order_katie   <dbl> 46, 127, 201, 178, 427, 341, 391, 25, 224, 374, 368, 32,~
## $ order_terri   <dbl> 58, 120, 119, 244, 278, 465, 379, 240, 236, 417, 459, 63~
## $ order_beth    <dbl> 201, 74, 147, 181, 393, 486, 408, 34, 163, 469, 314, 72,~
## $ order_naija   <dbl> 361, 438, 488, 460, 168, 140, 164, 443, 493, 175, 258, 3~
## $ order_erik    <dbl> 152, 6, 160, 236, 398, 377, 254, 69, 140, 409, 434, 233,~
## $ order_caitlin <dbl> 226, 459, 170, 464, 249, 70, 81, 478, 456, 5, 264, 409, ~
```

```r
# 3) Drop specified columns
records_df <- raw_data %>%
  dplyr::select(-dplyr::any_of(c("diagnosis", "assessment", "pathology")))

# 4) Filter for rows with non-empty values in ALL listed columns
required_cols <- c("nadia", "katie", "terri", "beth", "naija", "erik", "caitlin", "tracy")

records_df <- records_df %>%
  dplyr::filter(
    dplyr::if_all(
      dplyr::all_of(required_cols),
      ~ !is.na(.) &
        stringr::str_trim(as.character(.)) != "" &
        stringr::str_trim(as.character(.)) != "nan"
    )
  )

# 5) Report the updated row count
cat("\n=== After Dropping Columns + Filtering ===\n")
```

```
##
## === After Dropping Columns + Filtering ===
```

```r
cat("Updated rows:", nrow(records_df), "\n")
```

```
## Updated rows: 500
```

```r
# 6) Print the head of the dataframe
cat("\n=== Head of DataFrame ===\n")
```

```
##
## === Head of DataFrame ===
```

```r
print(head(records_df))
```

```
## # A tibble: 6 x 26
##   record_id tracy   nadia katie terri beth  naija erik  caitlin ai    time_tracy
##       <dbl> <chr>   <chr> <chr> <chr> <chr> <chr> <chr> <chr>   <lgl>      <dbl>
```

```
## 1    228755 "81500~ "815~ "815~ "815~ "815~ "815~ "815~ "81500~ FALSE       36.1
## 2     18284 "10249~ "102~ "102~ "102~ "102~ "102~ "102~ "10249~ FALSE       16.8
## 3    100014 "93886~ "414~ "414~ "372~ "432~ "414~ "414~ "41465~ FALSE      182.
## 4    258616 "12704~ "274~ "127~ "413~ "413~ "413~ "413~ "41360~ FALSE      211.
## 5     46602 "24079~ "240~ "240~ "240~ "240~ "240~ "240~ "24079~ TRUE        97.5
## 6     26645 "64379~ "389~ "600~ "389~ "389~ "665~ "261~ "66579~ TRUE       282.
## # i 15 more variables: time_nadia <dbl>, time_katie <dbl>, time_terri <dbl>,
## #   time_beth <dbl>, time_naija <dbl>, time_erik <dbl>, time_caitlin <dbl>,
## #   order_tracy <dbl>, order_nadia <dbl>, order_katie <dbl>, order_terri <dbl>,
## #   order_beth <dbl>, order_naija <dbl>, order_erik <dbl>, order_caitlin <dbl>
```

1.2 Reference Code Dictionary

Imports and cleans the master list of allowable codes used as the one-hot encoding reference.

```
# Build a stable numeric reference vector from the external code dictionary.
# Import reference codes
all_codes_df <- readr::read_csv("all_codes.csv", col_types = cols(.default = col_character()))

# Pull first column as the reference list (works whether the column is named or not)
all_codes <- all_codes_df %>%
  dplyr::pull(1) %>%
  stringr::str_trim() %>%
  .[. != "" & !is.na(.)] %>%
  suppressWarnings(as.numeric()) %>%  # convert valid code strings to numeric values
  .[!is.na(.)] %>%                    # remove entries that could not be parsed as codes
  unique()

# Optional: keep reference codes in a stable order
all_codes <- sort(all_codes)

cat("Number of Codes:", length(all_codes), "\n")
```

```
## Number of Codes: 7739
```

1.3 Data Reshaping

Reshapes coder-specific columns into one row per record and coder, then assigns each coder to an experience group.

```
# Standardize coder columns before pivoting them to a tidy long representation.
subjects <- c("nadia", "katie", "terri", "beth", "naija", "erik", "caitlin", "tracy")

records_long <- records_df %>%
  rename_with( ~ paste0("codes_", .x), .cols = any_of(subjects)) %>%
  pivot_longer(
    cols = matches("^(codes|time|order)_"),
    names_to = c(".value", "coder"),
    names_sep = "_"
  ) %>%
  mutate(
    group = case_when(
      coder %in% c("nadia", "katie", "tracy") ~ "clinicians",
      coder %in% c("terri", "beth") ~ "coders",
      coder %in% c("naija", "erik", "caitlin") ~ "students",
      TRUE ~ NA_character_
```

```
    )
  ) %>%
  select(record_id, ai, coder, group, codes, time, order) %>%
  arrange(record_id, match(coder, subjects))
```

1.4 Feature Engineering

Calculates character length from diagnosis and assessment text as a record-level covariate. Adds the record-level text-length covariate to the long-format coding data.

```
# Use source text from raw_data because diagnosis and assessment were removed from records_df.
record_lens <- raw_data %>%
  mutate(
    combined_text = paste(diagnosis, assessment, sep = " "),
    text_length   = nchar(combined_text)
  ) %>%
  select(c(record_id, text_length))

# Join the record-level covariate while retaining the current row order and all coder observations.
records_long <- records_long %>%
  dplyr::left_join(record_lens, by = "record_id")

head(records_long)
```

```
## # A tibble: 6 x 8
##   record_id ai    coder group      codes                  time order text_length
##       <dbl> <lgl> <chr> <chr>      <chr>                 <dbl> <dbl>       <int>
## 1      1022 TRUE  nadia clinicians "370466005\n40281500~  30.5   145         528
## 2      1022 TRUE  katie clinicians "370466005\n40281500~  77.6   292         528
## 3      1022 TRUE  terri coders     "370466005\n37198600~  58.8   439         528
## 4      1022 TRUE  beth  coders     "370466005\n40281500~  51.8   291         528
## 5      1022 TRUE  naija students   "370466005\n40281500~  33.4   283         528
## 6      1022 TRUE  erik  students   "370466005\n40281500~  34.7   360         528
```

Defines helpers for pairwise Jaccard similarity and the mean similarity across all coder pairs.

```
# Keep similarity calculations in reusable helpers so the grouped analysis remains readable.
# Function to calculate pairwise Jaccard score
jaccard_pairwise <- function(rater1, rater2) {
  return(jaccard::jaccard(rater1, rater2))
}

# Helper: average of all pairwise jaccards in a list of one-hot vectors
avg_pairwise_jaccard <- function(onehot_list) {
  # keep only non-null vectors
  onehot_list <- onehot_list[!vapply(onehot_list, is.null, logical(1))]

  n <- length(onehot_list)
  if (n < 2)
    return(NA_real_)

  pairs <- combn(n, 2)

  scores <- purrr::map_dbl(seq_len(ncol(pairs)), function(k) {
    v1 <- onehot_list[[pairs[1, k]]]
```

```
    v2 <- onehot_list[[pairs[2, k]]]
    s  <- jaccard_pairwise(v1, v2)

    # If both vectors are all-zeros, some implementations return NaN (0/0).
    # Common convention for Jaccard(empty, empty) is 1. Adjust if you prefer 0.
    if (is.nan(s))
      s <- 1
    s
  })

  mean(scores, na.rm = TRUE)
}
```

Defines a parser that converts newline-separated code strings into unique numeric code vectors.

```
# Normalize stored code text before it is encoded for agreement calculations.
# Convert newline-separated code text into a numeric vector
codes_text_to_numeric <- function(x, unique_codes = TRUE) {
  # Treat NA as empty
  if (is.na(x))
    return(numeric(0))

  # Coerce to character in case a factor slips in
  x <- as.character(x)

  # Trim overall string; if empty after trim, return empty vector
  x_trim <- stringr::str_trim(x)
  if (identical(x_trim, "") ||
      nchar(x_trim) == 0)
    return(numeric(0))

  # Split on any line break (\n, \r\n, \r)
  parts <- stringr::str_split(x_trim, pattern = "\\r?\\n|\\r")[[1]]

  # Trim each element and drop empties
  parts <- stringr::str_trim(parts)
  parts <- parts[parts != ""]

  if (length(parts) == 0)
    return(numeric(0))

  # Convert to numeric
  nums <- suppressWarnings(as.numeric(parts))

  # Drop any non-numeric entries (if present)
  nums <- nums[!is.na(nums)]

  if (unique_codes)
    nums <- unique(nums)

  nums
}
```

Defines a helper that maps each code vector to a binary vector aligned with the reference code list.

```r
# Encode observed codes against all_codes so every coder vector has identical length and order.
# One-hot encode a numeric code vector against the all_codes reference list
one_hot_encode_codes <- function(codes_vec, ref_codes = all_codes) {
  # Handle NULL / NA / empty
  if (is.null(codes_vec) ||
      length(codes_vec) == 0 || all(is.na(codes_vec))) {
    out <- integer(length(ref_codes))
    names(out) <- as.character(ref_codes)
    return(out)
  }

  # Ensure numeric + unique to avoid redundant indexing
  codes_vec <- unique(as.numeric(codes_vec))
  codes_vec <- codes_vec[!is.na(codes_vec)]

  out <- integer(length(ref_codes))
  names(out) <- as.character(ref_codes)

  # Efficient set membership via match (faster than %in% for large vectors)
  idx <- match(codes_vec, ref_codes, nomatch = 0L)
  idx <- idx[idx > 0L]
  if (length(idx) > 0L)
    out[idx] <- 1L

  out
}
```

Calculates average pairwise Jaccard agreement for every record and coder group, including missing record-group combinations.

```r
# Create a complete record-by-group agreement table for modeling and plotting.
# Define the groups you expect (so we can guarantee one row per record/group)
group_levels <- c("clinicians", "coders", "students")

# Compute avg pairwise jaccard per record/group
ai_by_record <- records_long %>%
  distinct(record_id, ai)

jaccard_by_record_group <- records_long %>%
  mutate(
    codes_vec = purrr::map(codes, codes_text_to_numeric),
    onehot    = purrr::map(codes_vec, one_hot_encode_codes, ref_codes = all_codes)
  ) %>%
  group_by(record_id, group) %>%
  summarise(
    ai = first(ai),
    # ai is constant within record
    jaccard_avg = avg_pairwise_jaccard(onehot),
    .groups = "drop"
  ) %>%
  # ensure one row per group per record_id
  right_join(
    tidyr::expand_grid(record_id = sort(unique(
      records_long$record_id
```

```
    )), group = group_levels),
    by = c("record_id", "group")
  ) %>%
  left_join(ai_by_record, by = "record_id", suffix = c("", ".ref")) %>%
  mutate(ai = coalesce(ai, ai.ref),
         group = factor(group, levels = group_levels)) %>%
  select(record_id, ai, group, jaccard_avg) %>%
  arrange(record_id, group)
```

Adds record text length to the record-by-group Jaccard analysis table.

```
# Join the text-length adjustment variable while retaining all rows in the Jaccard outcome table.
jaccard_by_record_group <- jaccard_by_record_group %>%
  dplyr::left_join(record_lens, by = "record_id")

jaccard_by_record_group
```

```
## # A tibble: 1,500 x 5
##    record_id ai    group      jaccard_avg text_length
##        <dbl> <lgl> <fct>            <dbl>       <int>
##  1      1022 TRUE  clinicians        0.5           528
##  2      1022 TRUE  coders            0.25          528
##  3      1022 TRUE  students          1             528
##  4      1028 TRUE  clinicians        1             732
##  5      1028 TRUE  coders            1             732
##  6      1028 TRUE  students          1             732
##  7      1727 TRUE  clinicians        1              49
##  8      1727 TRUE  coders            1              49
##  9      1727 TRUE  students          1              49
## 10      1993 TRUE  clinicians        1             319
## # i 1,490 more rows
```

Creates a coder-specific global task-order variable for the coding-time model.

```
# Construct sequential task order within coder for use as a fixed effect.
records_long <- records_long %>%
  group_by(coder) %>%
  arrange(order, .by_group = TRUE) %>%
  mutate(order_global = row_number()) %>%
  ungroup()
```

2. Exploratory Analysis

2.1 Exploratory Tables

Computes the average number of assigned codes for each coder group.

```
# Count whitespace-delimited code tokens, preserving missing values separately from empty strings.
avg_n_codes <- records_long %>%
  mutate(
    n_codes = case_when(
      is.na(codes) ~ NA_real_,
      str_trim(codes) == "" ~ 0,
      TRUE ~ str_count(str_trim(codes), "\\S+")
    )
  ) %>%
```

```
  group_by(group) %>%
  summarise(
    avg_n_codes = mean(n_codes, na.rm = TRUE),
    .groups = "drop"
  )

avg_n_codes
```

```
## # A tibble: 3 x 2
##   group      avg_n_codes
##   <chr>            <dbl>
## 1 clinicians        2.90
## 2 coders            3.18
## 3 students          2.44
```

Computes the average number of assigned codes for each condition.

```
# Count whitespace-delimited code tokens, preserving missing values separately from empty strings.
avg_n_codes <- records_long %>%
  mutate(
    n_codes = case_when(
      is.na(codes) ~ NA_real_,
      str_trim(codes) == "" ~ 0,
      TRUE ~ str_count(str_trim(codes), "\\S+")
    )
  ) %>%
  group_by(ai) %>%
  summarise(
    avg_n_codes = mean(n_codes, na.rm = TRUE),
    .groups = "drop"
  )

avg_n_codes
```

```
## # A tibble: 2 x 2
##   ai    avg_n_codes
##   <lgl>       <dbl>
## 1 FALSE        2.81
## 2 TRUE         2.78
```

Summarizes coding-time means and standard deviations by coder group.

```
# Produce an unadjusted descriptive summary of coding time by experience group.
records_long %>%
  group_by(group) %>%
  summarize(mean(time),
            sd(time))
```

```
## # A tibble: 3 x 3
##   group      `mean(time)` `sd(time)`
##   <chr>             <dbl>      <dbl>
## 1 clinicians        106.       104.
## 2 coders            105.       107.
## 3 students           63.3       56.5
```

Summarizes coding-time means and standard deviations by AI-use condition.

```r
# Produce an unadjusted descriptive summary of coding time by AI-use condition.
records_long %>%
  group_by(ai) %>%
  summarize(mean(time),
            sd(time))
```

```
## # A tibble: 2 x 3
##   ai    `mean(time)` `sd(time)`
##   <lgl>        <dbl>      <dbl>
## 1 FALSE        106.       101.
## 2 TRUE          72.7       79.1
```

Summarizes Jaccard means and standard deviations by coder group.

```r
# Produce an unadjusted descriptive summary of agreement by experience group.
jaccard_by_record_group %>%
  group_by(group) %>%
  summarize(mean(jaccard_avg),
            sd(jaccard_avg))
```

```
## # A tibble: 3 x 3
##   group      `mean(jaccard_avg)` `sd(jaccard_avg)`
##   <fct>                    <dbl>             <dbl>
## 1 clinicians               0.552             0.315
## 2 coders                   0.581             0.376
## 3 students                 0.565             0.332
```

Summarizes Jaccard means and standard deviations by AI-use condition.

```r
# Produce an unadjusted descriptive summary of agreement by AI-use condition.
jaccard_by_record_group %>%
  group_by(ai) %>%
  summarize(mean(jaccard_avg),
            sd(jaccard_avg))
```

```
## # A tibble: 2 x 3
##   ai    `mean(jaccard_avg)` `sd(jaccard_avg)`
##   <lgl>               <dbl>             <dbl>
## 1 FALSE               0.450             0.332
## 2 TRUE                0.682             0.311
```

2.2 AI Suggestion Acceptance

```r
# ------------------------------------------------------------------------
# 1. Import model predictions
#
# Expected JSON structure:
# {
#   "12345": ["111111", "222222"],
#   "12346": []
# }
# ------------------------------------------------------------------------

predictions_raw <- jsonlite::fromJSON(
  "gatortron_predictions.json",
  simplifyVector = FALSE
```

```
)

predictions_df <- tibble(
  record_id = as.integer(names(predictions_raw)),
  predicted_codes = unname(predictions_raw)
) %>%
  mutate(
    predicted_codes = map(
      predicted_codes,
      ~ sort(unique(as.character(.x)))
    )
  )

# ------------------------------------------------------------------------
# Robust code parser
# ------------------------------------------------------------------------

split_response_codes <- function(value) {

  # Each call should receive one dataframe cell
  if (length(value) == 0L || is.null(value) || all(is.na(value))) {
    return(character(0))
  }

  # Explicitly coerce factors, integers, and other atomic values to text
  value <- as.character(value[[1]])

  if (is.na(value) || trimws(value) == "") {
    return(character(0))
  }

  # Split on one or more whitespace characters
  result <- unlist(
    strsplit(trimws(value), split = "\\s+"),
    use.names = FALSE
  )

  # Remove blanks and duplicates, then sort for reproducibility
  sort(unique(result[nzchar(result)]))
}


# ------------------------------------------------------------------------
# Filter AI responses, parse response codes, and join predictions
# ------------------------------------------------------------------------

ai_comparison <- records_long %>%
  dplyr::filter(.data$ai %in% TRUE) %>%
  dplyr::mutate(
    record_id = as.integer(.data$record_id),
    group = as.character(.data$group),

    # Explicit package and column references prevent name collisions
```

```
    response_codes = purrr::map(
      as.character(.data$codes),
      split_response_codes
    )
  ) %>%
  dplyr::left_join(
    predictions_df,
    by = "record_id"
  )

# Stop if any AI-assisted responses do not have corresponding JSON predictions.
missing_prediction_ids <- ai_comparison %>%
  filter(map_lgl(predicted_codes, is.null)) %>%
  distinct(record_id) %>%
  pull(record_id)

if (length(missing_prediction_ids) > 0L) {
  stop(
    "No model predictions were found for these record_id values: ",
    paste(missing_prediction_ids, collapse = ", ")
  )
}

ai_comparison <- ai_comparison %>%
  mutate(
    # Number of unique codes submitted in each response
    n_response_codes = map_int(response_codes, length),

    # Number of submitted codes that were also AI predictions
    accepted_codes = map2(
      response_codes,
      predicted_codes,
      intersect
    ),
    n_accepted_codes = map_int(accepted_codes, length),

    # Exact set equality, independent of order
    perfect_match = map2_lgl(
      response_codes,
      predicted_codes,
      setequal
    )
  )


# ------------------------------------------------------------------------
# 5A. Overall metrics
#
# code_acceptance_pct denominator:
#   All unique codes submitted across AI-assisted responses
#
# perfect_match_pct denominator:
#   All AI-assisted responses
```

```r
# ------------------------------------------------------------------------

overall_metrics <- ai_comparison %>%
  summarise(
    n_responses = n(),

    total_submitted_codes = sum(n_response_codes),
    total_accepted_ai_codes = sum(n_accepted_codes),

    code_acceptance_pct = if_else(
      total_submitted_codes > 0,
      100 * total_accepted_ai_codes / total_submitted_codes,
      NA_real_
    ),

    n_perfect_matches = sum(perfect_match),
    perfect_match_pct = if_else(
      n_responses > 0,
      100 * n_perfect_matches / n_responses,
      NA_real_
    )
  )


# ------------------------------------------------------------------------
# 5B. Metrics by subject group
# ------------------------------------------------------------------------

group_metrics <- ai_comparison %>%
  group_by(group) %>%
  summarise(
    n_responses = n(),

    total_submitted_codes = sum(n_response_codes),
    total_accepted_ai_codes = sum(n_accepted_codes),

    code_acceptance_pct = if_else(
      total_submitted_codes > 0,
      100 * total_accepted_ai_codes / total_submitted_codes,
      NA_real_
    ),

    n_perfect_matches = sum(perfect_match),
    perfect_match_pct = if_else(
      n_responses > 0,
      100 * n_perfect_matches / n_responses,
      NA_real_
    ),

    .groups = "drop"
  )
```

```r
# ------------------------------------------------------------------------
# Optional: one presentation-ready results table
# ------------------------------------------------------------------------

results_table <- bind_rows(
  overall_metrics %>%
    mutate(group = "Overall", .before = 1),

  group_metrics
) %>%
  mutate(
    code_acceptance_pct = round(code_acceptance_pct, 1),
    perfect_match_pct = round(perfect_match_pct, 1)
  ) %>%
  select(
    group,
    n_responses,
    total_submitted_codes,
    total_accepted_ai_codes,
    code_acceptance_pct,
    n_perfect_matches,
    perfect_match_pct
  )

print(results_table)
```

```
## # A tibble: 4 x 7
##   group      n_responses total_submitted_codes total_accepted_ai_codes
##   <chr>            <int>                 <int>                   <int>
## 1 Overall           2000                  5550                    3636
## 2 clinicians         750                  2133                    1322
## 3 coders             500                  1533                     999
## 4 students           750                  1884                    1315
## # i 3 more variables: code_acceptance_pct <dbl>, n_perfect_matches <int>,
## #   perfect_match_pct <dbl>
```

2.3 Exploratory Plots

Creates and saves faceted histograms of record-level Jaccard agreement by AI use and coder group.

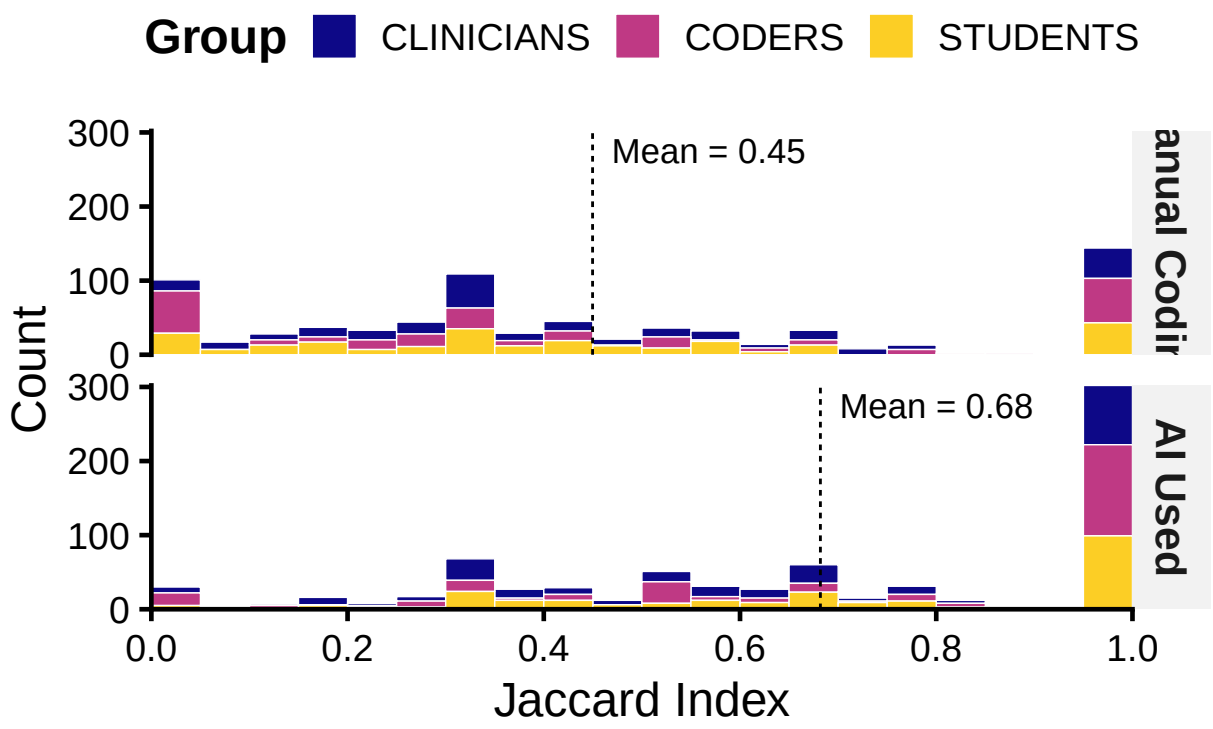
```r
# Prepare display labels, facet summaries, and the final agreement histogram.
jaccard_hist_df <- jaccard_by_record_group %>%
  rename(`AI Use` = ai, Group = group) %>%
  mutate(
    `AI Use` = case_when(
      `AI Use` == TRUE  ~ "AI Used",
      `AI Use` == FALSE ~ "Manual Coding",
      TRUE              ~ NA_character_
    ),
    Group = toupper(Group),
    # Nice ordering & stable legend
    `AI Use` = factor(`AI Use`, levels = c("Manual Coding", "AI Used"))
  ) %>%
  filter(!is.na(`AI Use`), !is.na(Group), !is.na(jaccard_avg))
```

```r
jaccard_vline_df <- jaccard_hist_df %>%
  group_by(`AI Use`) %>%
  summarise(mean_jaccard = mean(jaccard_avg, na.rm = TRUE),
            .groups = "drop")

p_jaccard_hist <- ggplot(jaccard_hist_df, aes(x = jaccard_avg, fill = Group)) +
  geom_histogram(
    binwidth = 0.05,
    boundary = 0,
    closed = "left",
    color = "white",
    linewidth = 0.25
  ) +
  geom_vline(
    data = jaccard_vline_df,
    aes(xintercept = mean_jaccard),
    color = "black",
    linetype = "22",
    # clean dashed
    linewidth = 0.5
  ) +
  geom_text(
    data = jaccard_vline_df,
    aes(
      x = mean_jaccard,
      y = Inf,
      label = paste0("Mean = ", round(mean_jaccard, 2)),
      hjust = ifelse(mean_jaccard > 0.8, 1.1, -0.1)  # shift left if near 1
    ),
    vjust = 1.3,
    inherit.aes = FALSE,
    size = 4.5
  ) +
  facet_grid(`AI Use` ~ .) +
  coord_cartesian(xlim = c(0, 1), expand = FALSE) +
  scale_x_continuous(
    breaks = seq(0, 1, 0.2),
    labels = function(x)
      ifelse(x %in% c(0, 1), sprintf("%.1f", x), sprintf("%.1f", x))
  ) +
  scale_fill_viridis_d(end = 0.9, option = "C") +
  labs(
    title = "Inter-Coder Reliability",
    # subtitle = "per record x group",
    # subtitle = "Faceted by AI usage; dashed line indicates mean within facet",
    x = "Jaccard Index",
    y = "Count",
    fill = "Group"
  ) +
  theme_classic(base_size = 18) +
  theme(
    plot.title.position = "plot",
    plot.title = element_text(face = "bold", size = 22),
```

```
    plot.subtitle = element_text(size = 16, margin = margin(b = 10)),
    axis.title = element_text(size = 18),
    axis.text  = element_text(size = 14, color = "black"),
    strip.text = element_text(face = "bold", size = 16),
    strip.background = element_rect(fill = "grey95", color = NA),
    legend.position = "top",
    legend.title = element_text(face = "bold"),
    legend.text = element_text(size = 14),
    panel.grid = element_blank(),
    panel.spacing.x = unit(0.6, "lines"),
    # small increase only
    panel.spacing.y = unit(0.8, "lines"),
    axis.text.x = element_text(margin = margin(t = 5))
  )

p_jaccard_hist
```




```
ggsave(
  "../plots/jaccard_hist.png",
  plot = p_jaccard_hist,
  width = 8,
  height = 8,
  # <-- increase this
  dpi = 300
)
```

Creates and saves a faceted coding-time trend plot by AI use and coder group.

```r
# Re-index task order within coder and AI condition before plotting coding-time trajectories.
coding_speed_df <- records_long %>%
  group_by(coder, ai) %>%
  arrange(order, .by_group = TRUE) %>%   # preserves the intended sequence
  mutate(order = row_number()) %>%       # re-index within coder × ai
  ungroup() %>%
  rename(`AI Use` = ai) %>%
  mutate(
    `AI Use` = case_when(
      `AI Use` == TRUE  ~ "AI Used",
      `AI Use` == FALSE ~ "Manual Coding",
      TRUE              ~ NA_character_
    )
  ) %>%
  rename(Group = group) %>%
  mutate(Group = toupper(Group)) %>%
  mutate(`AI Use` = factor(`AI Use`, levels = c("Manual Coding", "AI Used")))

coding_speed_hline_df <- coding_speed_df %>%
  group_by(`AI Use`) %>%
  summarise(mean_time = mean(time , na.rm = TRUE))

p2 <- ggplot(coding_speed_df, aes(x = order, y = time, color = Group)) +
  geom_point(alpha = 0.2, size = 1.2) +
  geom_smooth(method = "lm",
              se = FALSE,
              linewidth = 1) +
  geom_hline(
    data = coding_speed_hline_df,
    aes(yintercept = mean_time),
    color = "black",
    linetype = "22",
    linewidth = 0.5
  ) +
  geom_text(
    data = coding_speed_hline_df,
    aes(
      x = Inf,
      y = mean_time,
      label = paste0("Mean = ", round(mean_time, 1), " s")
    ),
    hjust = 1.1,
    # pull slightly inside right edge
    vjust = -0.4,
    # place just above line
    inherit.aes = FALSE,
    size = 4.5
  ) +
  facet_grid( ~ `AI Use`) +
  scale_y_continuous(
    trans = scales::log_trans(base = exp(1)),
    #  better than log(time) in aes
    labels = label_number(accuracy = 1)
```

```
  ) +
  scale_color_viridis_d(end = 0.9, option = "C") +
  labs(
    title = "Coding Speed Over Time",
    # subtitle = "Log-scaled time; dashed line indicates mean within facet",
    x = "Coding Order",
    y = "Coding Time [seconds, log scale]",
    color = "Group"
  ) +
  theme_classic(base_size = 18) +
  theme(
    plot.title.position = "plot",
    plot.title = element_text(face = "bold", size = 22),
    plot.subtitle = element_text(size = 16, margin = margin(b = 10)),

    axis.title = element_text(size = 18),
    axis.text  = element_text(size = 14, color = "black"),

    strip.text = element_text(face = "bold", size = 16),
    strip.background = element_rect(fill = "grey95", color = NA),

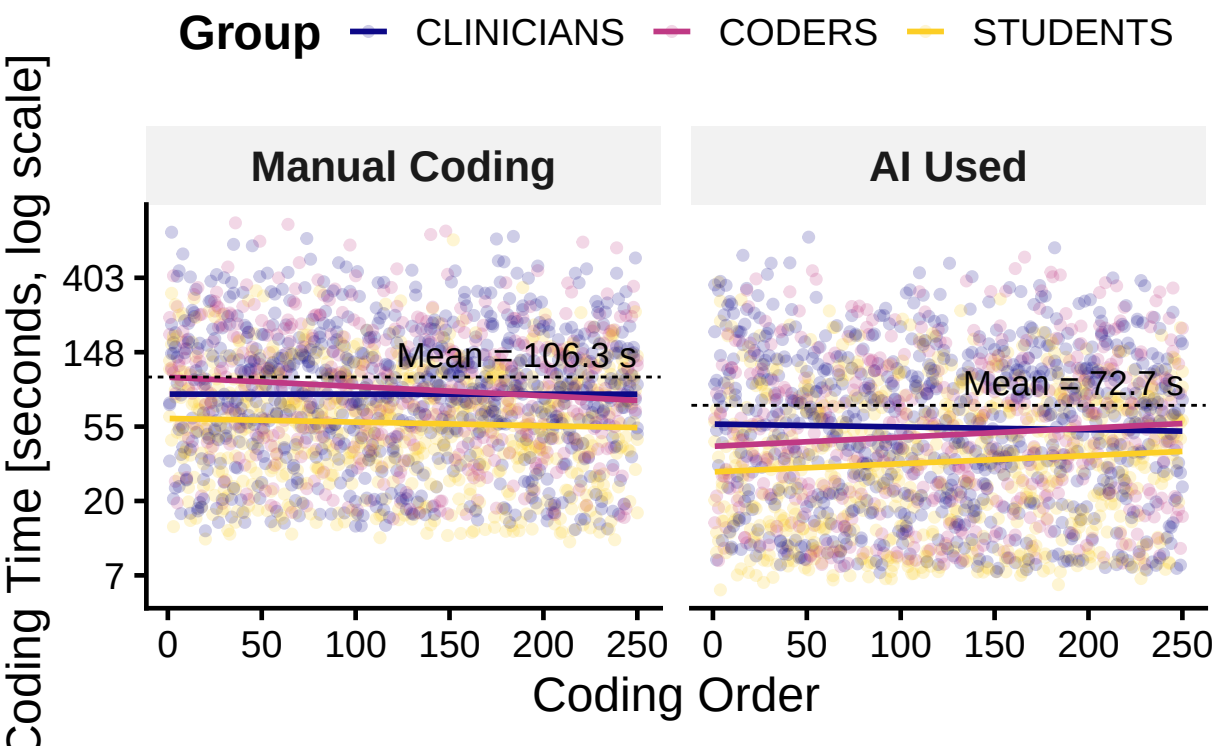
    legend.position = "top",
    legend.title = element_text(face = "bold"),
    legend.text = element_text(size = 14),

    panel.grid = element_blank(),
    panel.spacing.x = unit(0.8, "lines")  # matches your histogram tuning
  )

p2
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

Coding Speed Over Time



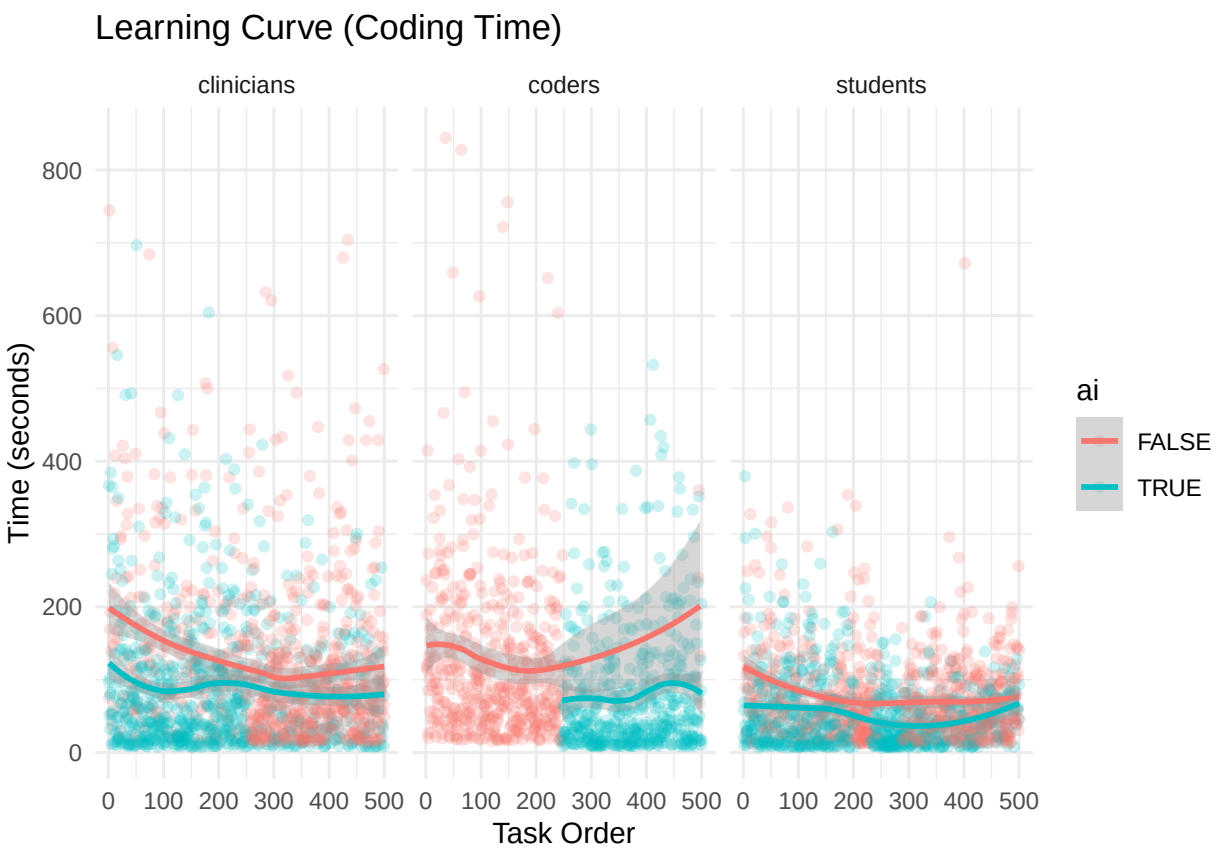

```
ggsave(
  "../plots/time_plot.png",
  plot = p2,
  width = 8,
  height = 8,
  # <-- increase this
  dpi = 300
)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

Creates an exploratory LOESS learning-curve plot using the coder-level global task order.

```
# Explore a flexible coding-time trend across global task order.
ggplot(records_long,
       aes(x = order_global, y = time, color = ai)) +
  geom_point(alpha = 0.2) +
  geom_smooth(method = "loess") +
  facet_wrap(~ group) +
  labs(
    title = "Learning Curve (Coding Time)",
    x = "Task Order",
    y = "Time (seconds)"
  ) +
  theme_minimal()
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



3. Jaccard Agreement Model

Moves Jaccard values away from exact zero and one so they can be modeled with a beta distribution.

```
# Apply a standard boundary adjustment required for beta outcomes at 0 or 1.
n <- nrow(jaccard_by_record_group)

jaccard_by_record_group <- jaccard_by_record_group %>%
  mutate(jaccard_beta = (jaccard_avg * (n - 1) + 0.5) / n)
```

Fits a beta mixed-effects model for Jaccard agreement with AI use, group, text length, and record-level random intercepts.

```
# Model agreement while accounting for repeated group observations within records.
fit_beta <- glmmTMB(
  jaccard_beta ~ ai * group + text_length + (1 | record_id),
  family = beta_family(),
  data = jaccard_by_record_group
)
```

```
## Warning in (function (start, objective, gradient = NULL, hessian = NULL, :
## NA/NaN function evaluation
## Warning in (function (start, objective, gradient = NULL, hessian = NULL, :
## NA/NaN function evaluation
## Warning in (function (start, objective, gradient = NULL, hessian = NULL, :
## NA/NaN function evaluation
```

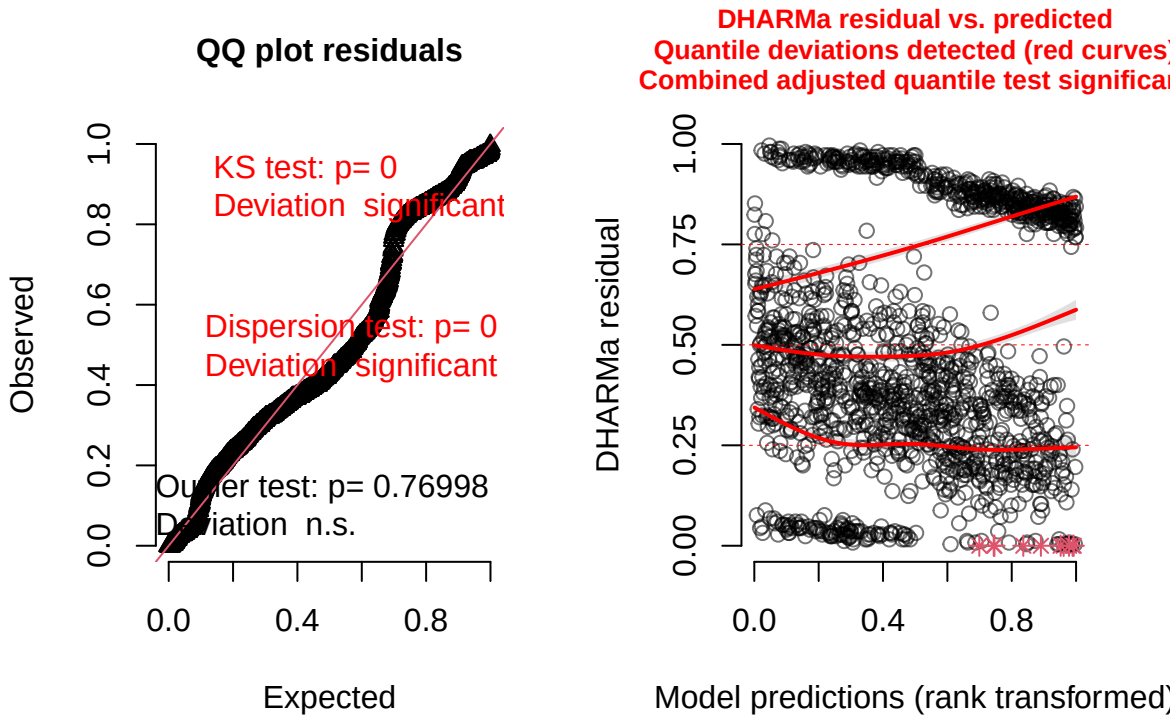
```
summary(fit_beta)
```

```
##  Family: beta  ( logit )
## Formula:          jaccard_beta ~ ai * group + text_length + (1 | record_id)
## Data: jaccard_by_record_group
##
##       AIC       BIC    logLik -2*log(L)  df.resid
##   -3690.7   -3642.9    1854.3   -3708.7      1491
##
## Random effects:
##
## Conditional model:
##  Groups    Name        Variance Std.Dev.
##  record_id (Intercept) 1.178    1.085
## Number of obs: 1500, groups:  record_id, 500
##
## Dispersion parameter for beta family ():  1.4
##
## Conditional model:
##                         Estimate Std. Error z value Pr(>|z|)
## (Intercept)           3.350e-01  1.334e-01   2.512 0.012022 *
## aiTRUE                7.306e-01  1.466e-01   4.985 6.18e-07 ***
## groupcoders          -1.940e-01  1.043e-01  -1.860 0.062872 .
## groupstudents        -1.418e-01  1.081e-01  -1.312 0.189366
## text_length          -1.456e-04  5.011e-05  -2.906 0.003656 **
## aiTRUE:groupcoders    5.300e-01  1.487e-01   3.564 0.000365 ***
## aiTRUE:groupstudents  3.641e-01  1.514e-01   2.405 0.016182 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

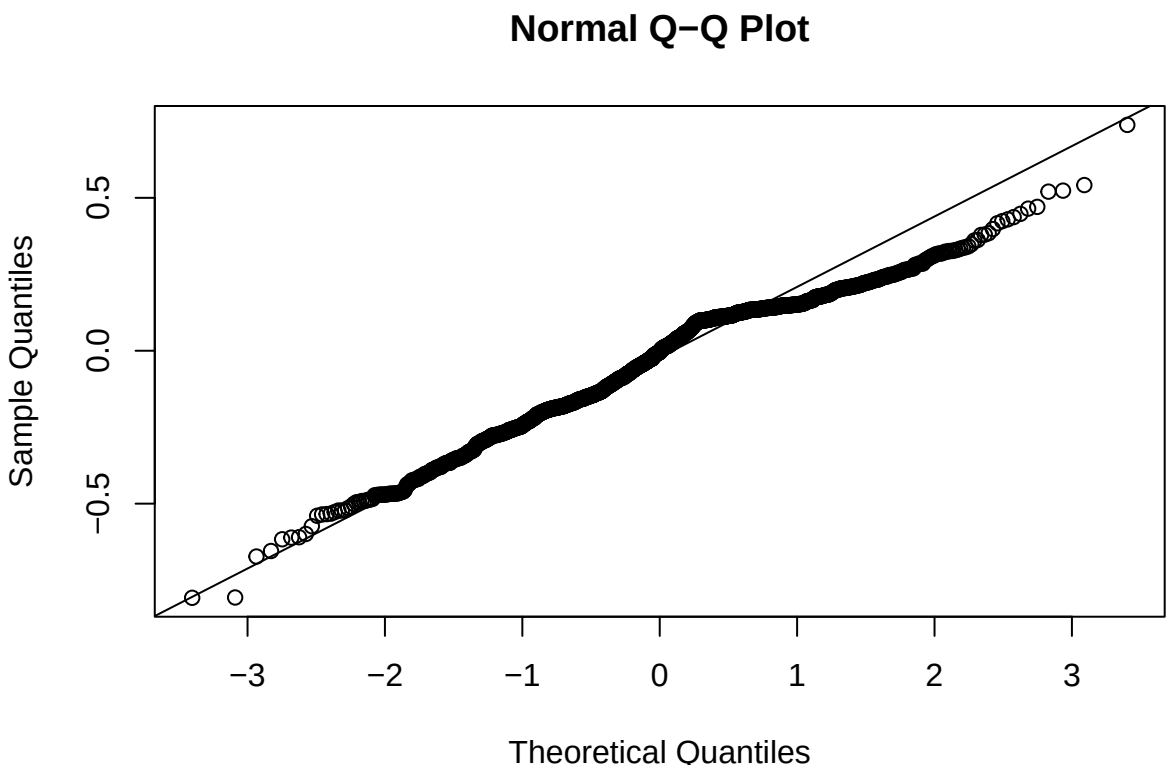
4.1 Jaccard Model Diagnostics

Evaluates the beta model with simulated residuals, residual plots, and model-fit summaries.

```
# Inspect residual behavior and summarize explanatory performance for the fitted beta model.
sim_beta <- simulateResiduals(fit_beta)
plot(sim_beta)
```




```
qqnorm(residuals(fit_beta))
qqline(residuals(fit_beta))
```

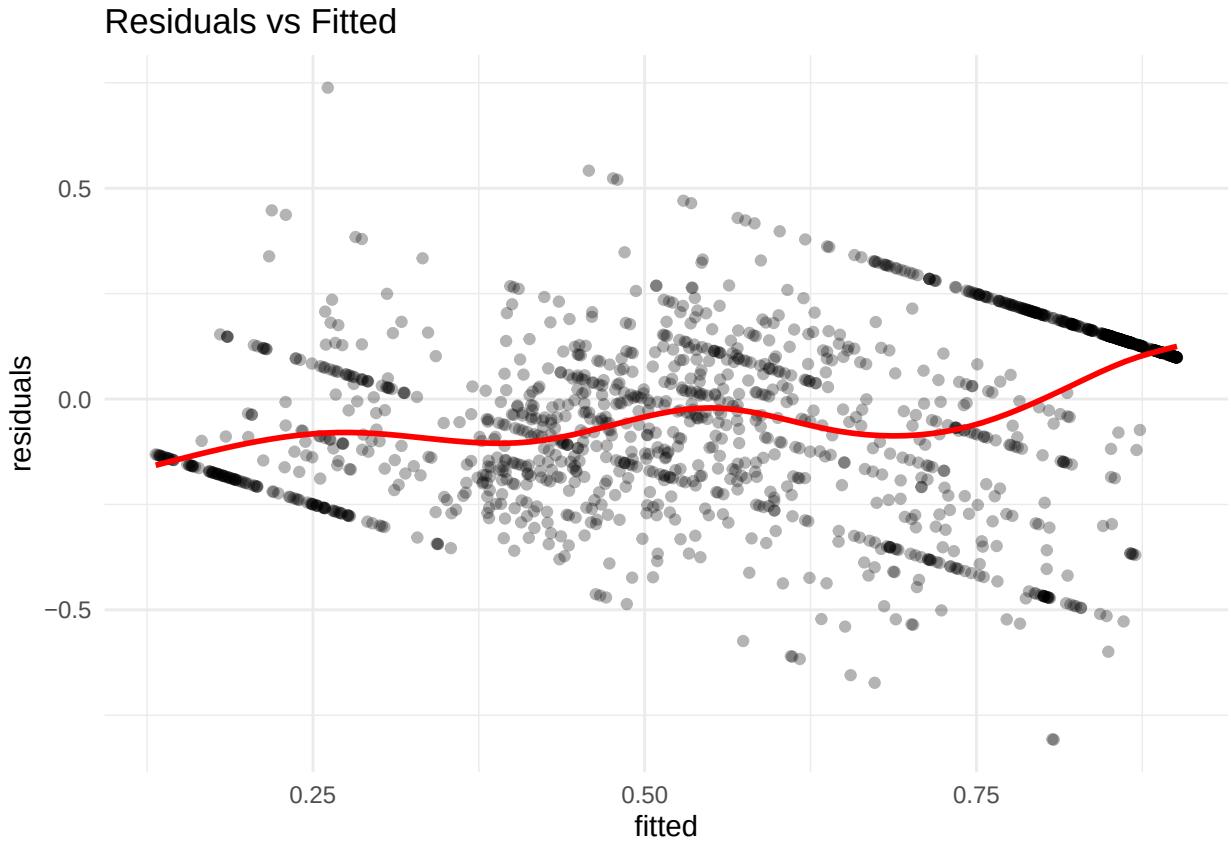



```
beta_residuals_df <- data.frame(
  fitted = fitted(fit_beta),
```

```
  residuals = residuals(fit_beta)
)

ggplot(beta_residuals_df, aes(x = fitted, y = residuals)) +
  geom_point(alpha = 0.3) +
  geom_smooth(se = FALSE, color = "red") +
  labs(title = "Residuals vs Fitted") +
  theme_minimal()
```

```
## `geom_smooth()` using method = 'gam' and formula = 'y ~ s(x, bs = "cs")'
```




```
performance::model_performance(fit_beta)
```

```
## # Indices of model performance
##
## AIC     |    AICc |     BIC | R2 (cond.) | R2 (marg.) |   ICC |  RMSE | Sigma
## ----------------------------------------------------------------------------
## -3690.7 | -3690.6 | -3642.9 |      0.869 |      0.188 | 0.839 | 0.206 | 1.403
```

```
# R2 (comparable metric)
r2(fit_beta)
```

```
## # R2 for Mixed Models
##
##   Conditional R2: 0.869
##      Marginal R2: 0.188
```

4.2 Jaccard Model Inference

Calculates estimated marginal means and Tukey-adjusted pairwise contrasts for the AI-by-group interaction.

```
# Estimate all AI-use and coder-group combinations on the response scale.
emm_beta <- emmeans(fit_beta, ~ ai * group, type = "response")

pairs(emm_beta, adjust = "tukey")
```

```
##  contrast                            odds.ratio     SE  df null z.ratio p.value
##  FALSE clinicians / TRUE clinicians      0.482 0.0706 Inf    1  -4.985 <0.0001
##  FALSE clinicians / FALSE coders         1.214 0.1270 Inf    1   1.860  0.4270
##  FALSE clinicians / TRUE coders          0.344 0.0502 Inf    1  -7.312 <0.0001
##  FALSE clinicians / FALSE students       1.152 0.1250 Inf    1   1.312  0.7783
##  FALSE clinicians / TRUE students        0.386 0.0565 Inf    1  -6.509 <0.0001
##  TRUE clinicians / FALSE coders          2.521 0.3600 Inf    1   6.475 <0.0001
##  TRUE clinicians / TRUE coders           0.715 0.0757 Inf    1  -3.173  0.0188
##  TRUE clinicians / FALSE students        2.393 0.3480 Inf    1   5.991 <0.0001
##  TRUE clinicians / TRUE students         0.801 0.0849 Inf    1  -2.097  0.2886
##  FALSE coders / TRUE coders              0.283 0.0403 Inf    1  -8.866 <0.0001
##  FALSE coders / FALSE students           0.949 0.0975 Inf    1  -0.508  0.9959
##  FALSE coders / TRUE students            0.318 0.0453 Inf    1  -8.038 <0.0001
##  TRUE coders / FALSE students            3.349 0.4860 Inf    1   8.332 <0.0001
##  TRUE coders / TRUE students             1.121 0.1170 Inf    1   1.086  0.8873
##  FALSE students / TRUE students          0.335 0.0487 Inf    1  -7.523 <0.0001
##
## P value adjustment: tukey method for comparing a family of 6 estimates
## Tests are performed on the log odds ratio scale
```

```
confint(pairs(emm_beta))
```

```
##  contrast                            odds.ratio     SE  df asymp.LCL asymp.UCL
##  FALSE clinicians / TRUE clinicians      0.482 0.0706 Inf     0.317     0.731
##  FALSE clinicians / FALSE coders         1.214 0.1270 Inf     0.902     1.634
##  FALSE clinicians / TRUE coders          0.344 0.0502 Inf     0.227     0.522
##  FALSE clinicians / FALSE students       1.152 0.1250 Inf     0.847     1.568
##  FALSE clinicians / TRUE students        0.386 0.0565 Inf     0.254     0.585
##  TRUE clinicians / FALSE coders          2.521 0.3600 Inf     1.678     3.787
##  TRUE clinicians / TRUE coders           0.715 0.0757 Inf     0.528     0.966
##  TRUE clinicians / FALSE students        2.393 0.3480 Inf     1.580     3.624
##  TRUE clinicians / TRUE students         0.801 0.0849 Inf     0.592     1.083
##  FALSE coders / TRUE coders              0.283 0.0403 Inf     0.189     0.425
##  FALSE coders / FALSE students           0.949 0.0975 Inf     0.708     1.272
##  FALSE coders / TRUE students            0.318 0.0453 Inf     0.212     0.477
##  TRUE coders / FALSE students            3.349 0.4860 Inf     2.215     5.062
##  TRUE coders / TRUE students             1.121 0.1170 Inf     0.831     1.511
##  FALSE students / TRUE students          0.335 0.0487 Inf     0.221     0.507
##
## Confidence level used: 0.95
## Conf-level adjustment: tukey method for comparing a family of 6 estimates
## Intervals are back-transformed from the log odds ratio scale
```

Prints the response-scale estimated marginal means for each AI-by-group combination.

```
# Display the interaction-level marginal mean estimates used for interpretation.
summary(emm_beta)
```

```
##  ai    group      response     SE  df asymp.LCL asymp.UCL
##  FALSE clinicians    0.523 0.0258 Inf     0.472     0.573
##   TRUE clinicians    0.695 0.0220 Inf     0.650     0.736
##  FALSE coders        0.475 0.0244 Inf     0.427     0.523
##   TRUE coders        0.761 0.0186 Inf     0.723     0.796
##  FALSE students      0.488 0.0255 Inf     0.438     0.538
##   TRUE students      0.740 0.0199 Inf     0.699     0.777
##
## Confidence level used: 0.95
## Intervals are back-transformed from the logit scale
```

Calculates response-scale marginal means and contrasts for AI use averaged across groups.

```
# Test the average AI-use contrast across coder groups.
emm_beta_ai <- emmeans(fit_beta, ~ ai, type = "response")
```

```
## NOTE: Results may be misleading due to involvement in interactions
```

```
summary(emm_beta_ai)
```

```
##  ai    response     SE  df asymp.LCL asymp.UCL
##  FALSE    0.495 0.0202 Inf     0.456     0.535
##   TRUE    0.733 0.0163 Inf     0.700     0.764
##
## Results are averaged over the levels of: group
## Confidence level used: 0.95
## Intervals are back-transformed from the logit scale
```

```
pairs(emm_beta_ai)
```

```
##  contrast     odds.ratio     SE  df null z.ratio p.value
##  FALSE / TRUE      0.357 0.0416 Inf    1  -8.834 <0.0001
##
## Results are averaged over the levels of: group
## Tests are performed on the log odds ratio scale
```

```
confint(pairs(emm_beta_ai))
```

```
##  contrast     odds.ratio     SE  df asymp.LCL asymp.UCL
##  FALSE / TRUE      0.357 0.0416 Inf     0.285     0.449
##
## Results are averaged over the levels of: group
## Confidence level used: 0.95
## Intervals are back-transformed from the log odds ratio scale
```

Calculates response-scale marginal means and contrasts for coder group averaged across AI conditions.

```
# Test pairwise coder-group differences averaged across AI-use conditions.
emm_beta_group <- emmeans(fit_beta, ~ group, type = "response")
```

```
## NOTE: Results may be misleading due to involvement in interactions
```

```
summary(emm_beta_group)
```

```
##  group      response     SE  df asymp.LCL asymp.UCL
##  clinicians    0.613 0.0174 Inf     0.578     0.646
##  coders        0.629 0.0165 Inf     0.596     0.661
##  students      0.622 0.0170 Inf     0.588     0.655
##
```

```
## Results are averaged over the levels of: ai
## Confidence level used: 0.95
## Intervals are back-transformed from the logit scale
```

```
pairs(emm_beta_group)
```

```
##  contrast              odds.ratio     SE  df null z.ratio p.value
##  clinicians / coders        0.931 0.0692 Inf    1  -0.956  0.6045
##  clinicians / students      0.961 0.0727 Inf    1  -0.532  0.8558
##  coders / students          1.031 0.0757 Inf    1   0.420  0.9074
##
## Results are averaged over the levels of: ai
## P value adjustment: tukey method for comparing a family of 3 estimates
## Tests are performed on the log odds ratio scale
```

4.3 Jaccard Model Visualization

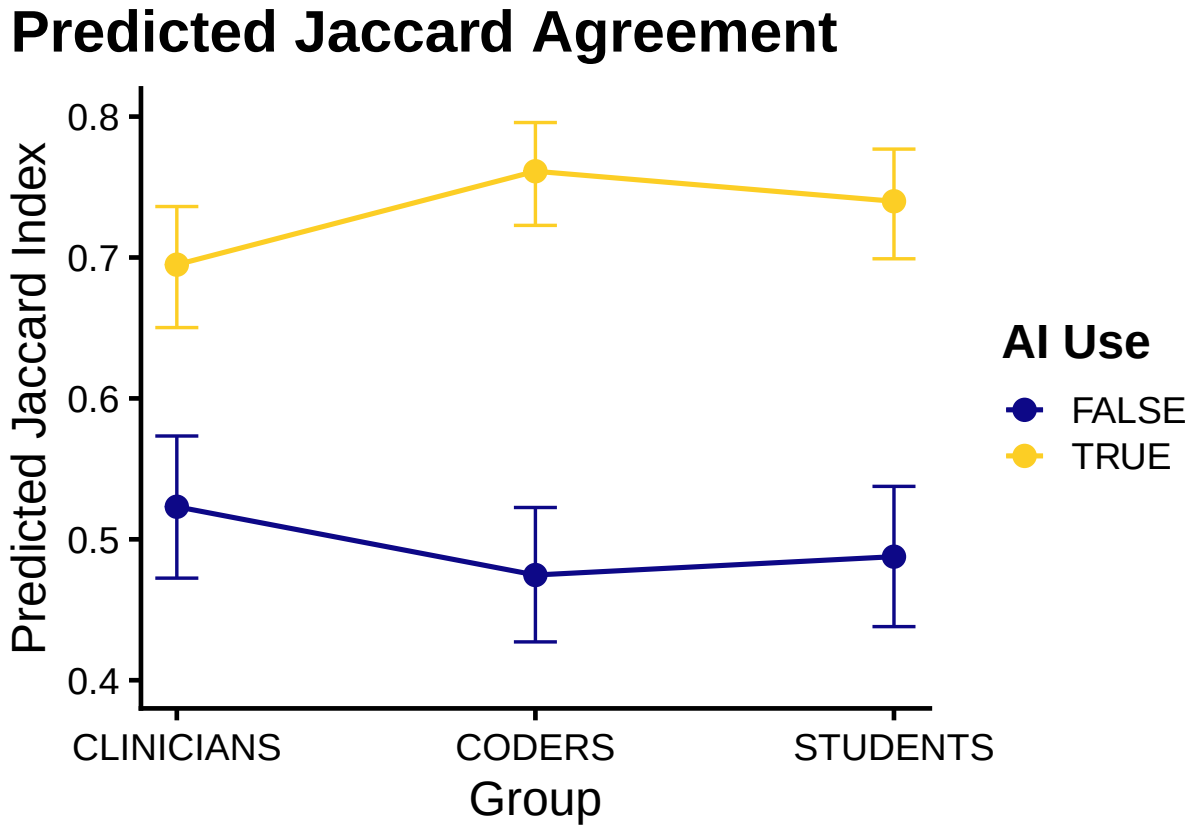
Plots and saves predicted Jaccard agreement and confidence intervals by group and AI use.

```
# Convert response-scale estimated marginal means to a plotting data frame.
# glmmTMB uses asymptotic confidence-limit names, which are standardized here
# so the plotting code does not depend on model-specific column labels.
plot_beta <- as.data.frame(summary(emm_beta, type = "response")) %>%
  transmute(
    ai,
    group = toupper(group),
    response,
    lower.CL = asymp.LCL,
    upper.CL = asymp.UCL
  )

p_jaccard_emmeans <- ggplot(
  plot_beta,
  aes(x = group, y = response, color = ai, group = ai)
) +
  # CI bars first so lines/points sit on top
  geom_errorbar(
    aes(ymin = lower.CL, ymax = upper.CL),
    width = 0.12,
    linewidth = 0.6
  ) +
  geom_line(
    linewidth = 0.9
  ) +
  geom_point(
    size = 3.2
  ) +
  # Optional: keep within Jaccard bounds (0-1) without chopping CIs off harshly
  # coord_cartesian(ylim = c(0.4, 0.8), expand = FALSE) +
  scale_x_discrete(expand = expansion(add = 0.1)) +  # slight buffer (not 0 to avoid clipping)
  coord_cartesian(ylim = c(0.4, 0.8), clip = "off") +
  theme(plot.margin = margin(10, 20, 10, 10)) +
  scale_y_continuous(
    breaks = seq(0, 1, 0.1),
    labels = function(x) sprintf("%.1f", x)
```

```
  ) +
  scale_color_viridis_d(end = 0.9, option = "C") +
  labs(
    title = "Predicted Jaccard Agreement",
    x = "Group",
    y = "Predicted Jaccard Index",
    color = "AI Use"
  ) +
  theme_classic(base_size = 18) +
  theme(
    plot.title.position = "plot",
    plot.title = element_text(face = "bold", size = 22),
    plot.subtitle = element_text(size = 16, margin = margin(b = 10)),
    axis.title = element_text(size = 18),
    axis.text  = element_text(size = 14, color = "black"),
    legend.position = "right",
    legend.title = element_text(face = "bold"),
    legend.text = element_text(size = 14),
    panel.grid = element_blank(),
    axis.text.x = element_text(margin = margin(t = 5))
  )

p_jaccard_emmeans
```




```
ggsave(
  "../plots/jaccard_emmeans.png",
  plot = p_jaccard_emmeans,
```

```
  width = 8,
  height = 6,
  dpi = 300
)
```

4. Coding Time Model

Fits a linear mixed-effects model for log coding time with coder and record random intercepts.

```
# Model log-transformed duration while accounting for repeated observations by coder and record.
# Log transformation requires every modeled coding-time value to be strictly positive.
if (any(records_long$time <= 0, na.rm = TRUE)) {
  stop("Coding time contains zero or negative values and cannot be log-transformed.")
}

fit_time <- lmer(
  log(time) ~ ai * group + order_global + text_length +
    (1 | coder) + (1 | record_id),
  data = records_long,
  REML = TRUE
)
```

```
## Warning: Some predictor variables are on very different scales: consider
## rescaling
## Warning: Some predictor variables are on very different scales: consider
## rescaling
```

```
summary(fit_time)
```

```
## Linear mixed model fit by REML. t-tests use Satterthwaite's method [
## lmerModLmerTest]
## Formula: log(time) ~ ai * group + order_global + text_length + (1 | coder) +
##     (1 | record_id)
##    Data: records_long
##
## REML criterion at convergence: 5573
##
## Scaled residuals:
##     Min      1Q  Median      3Q     Max
## -4.1916 -0.5828 -0.0210  0.5505  4.6015
##
## Random effects:
##  Groups    Name        Variance Std.Dev.
##  record_id (Intercept) 0.51659  0.7187
##  coder     (Intercept) 0.02225  0.1492
##  Residual              0.15218  0.3901
## Number of obs: 4000, groups:  record_id, 500; coder, 8
##
## Fixed effects:
##                          Estimate Std. Error         df t value Pr(>|t|)
## (Intercept)           4.182e+00  1.087e-01  1.230e+01  38.480 3.42e-14 ***
## aiTRUE               -4.829e-01  6.760e-02  5.611e+02  -7.143 2.85e-12 ***
## groupcoders          -8.468e-02  1.383e-01  5.173e+00  -0.612   0.5662
## groupstudents        -4.033e-01  1.235e-01  5.137e+00  -3.267   0.0214 *
## order_global         -9.460e-04  5.001e-05  3.505e+03 -18.916  < 2e-16 ***
```

```
## text_length           3.051e-04  2.517e-05  4.970e+02  12.123  < 2e-16 ***
## aiTRUE:groupcoders    1.430e-01  3.586e-02  3.493e+03   3.989 6.76e-05 ***
## aiTRUE:groupstudents -3.897e-02  2.853e-02  3.490e+03  -1.366   0.1721
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Correlation of Fixed Effects:
##             (Intr) aiTRUE grpcdr grpstd ordr_g txt_ln aTRUE:grpc
## aiTRUE      -0.340
## groupcoders -0.515  0.025
## groupstdnts -0.569  0.025  0.447
## order_globl -0.132  0.061  0.060  0.006
## text_length -0.402  0.056  0.000  0.000 -0.007
## aTRUE:grpcd  0.113 -0.196 -0.130 -0.049 -0.459  0.003
## aTRUE:grpst  0.073 -0.214 -0.055 -0.116 -0.055  0.000  0.422
## fit warnings:
## Some predictor variables are on very different scales: consider rescaling
```

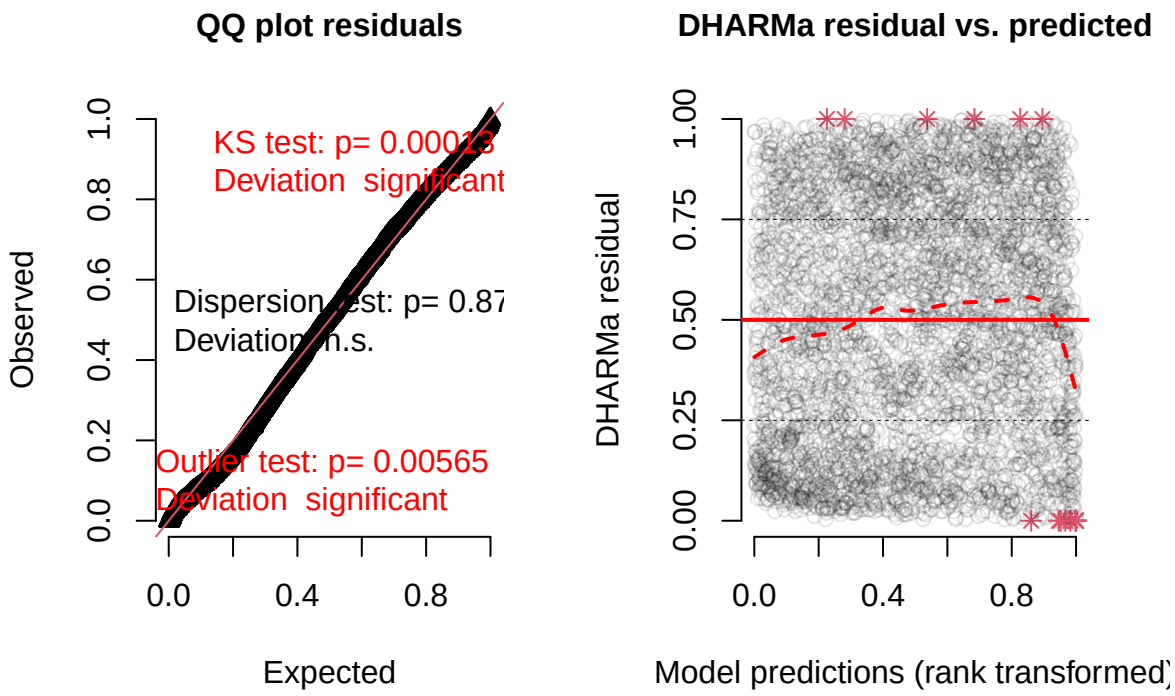
```
anova(fit_time, type = 3)
```

```
## Type III Analysis of Variance Table with Satterthwaite's method
##              Sum Sq Mean Sq NumDF  DenDF  F value    Pr(>F)
## ai            7.095   7.095     1  499.0  46.6237 2.507e-11 ***
## group         2.206   1.103     2    5.0   7.2488    0.0333 *
## order_global 54.451  54.451     1 3505.3 357.8062 < 2.2e-16 ***
## text_length  22.366  22.366     1  497.0 146.9734 < 2.2e-16 ***
## ai:group      4.143   2.072     2 3491.7  13.6134 1.290e-06 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

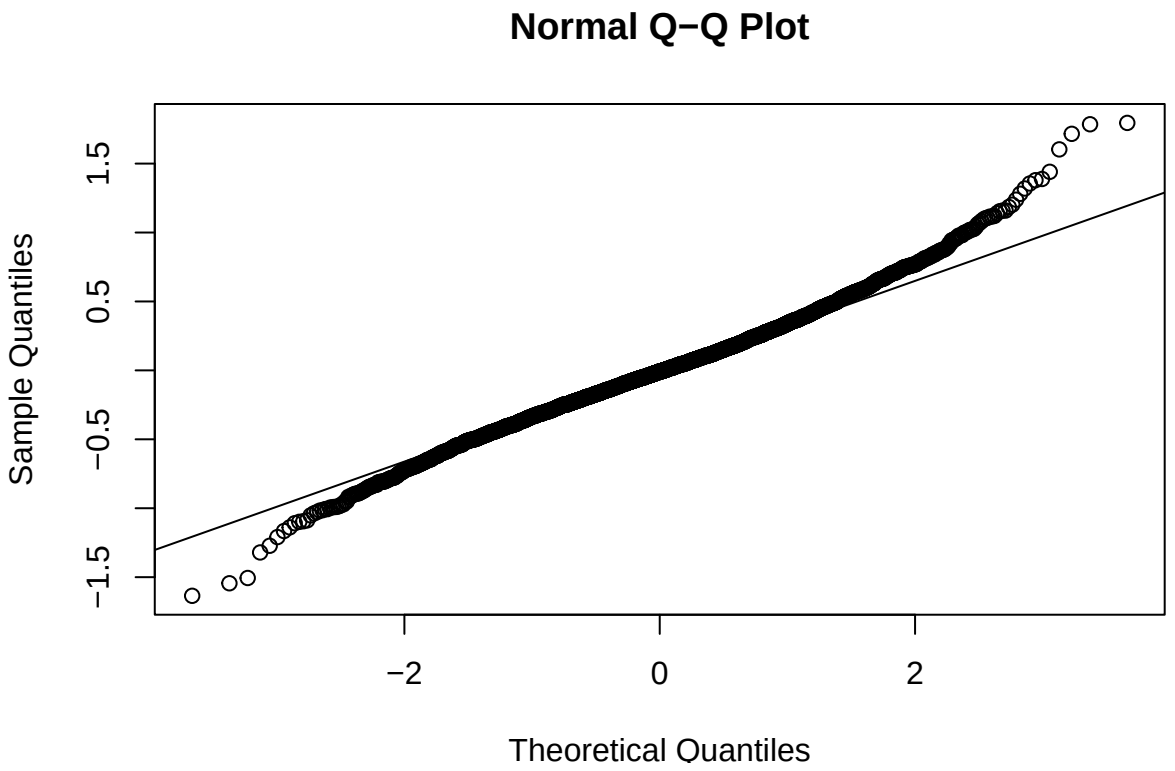
5.1 Coding Time Model Diagnostics

Evaluates the coding-time model with residual diagnostics and model-fit summaries.

```
# Inspect duration-model residuals
sim_time <- simulateResiduals(fit_time)
plot(sim_time)
```




```
qqnorm(residuals(fit_time))
qqline(residuals(fit_time))
```

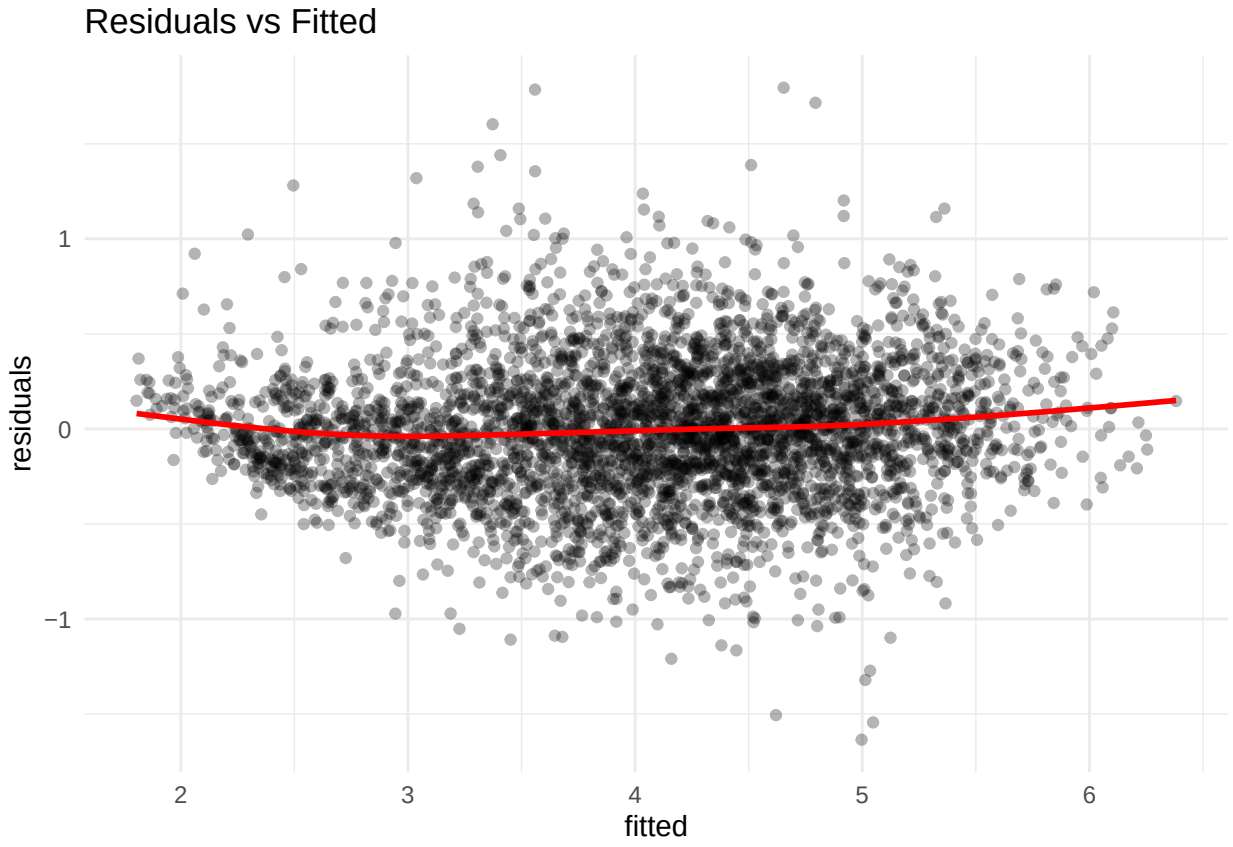



```
time_residuals_df <- data.frame(
  fitted = fitted(fit_time),
```

```
  residuals = residuals(fit_time)
)

ggplot(time_residuals_df, aes(x = fitted, y = residuals)) +
  geom_point(alpha = 0.3) +
  geom_smooth(se = FALSE, color = "red") +
  labs(title = "Residuals vs Fitted") +
  theme_minimal()
```

```
## `geom_smooth()` using method = 'gam' and formula = 'y ~ s(x, bs = "cs")'
```




```
performance::model_performance(fit_time)
```

```
## Warning: Log-likelihood is corrected for models with transformed response.
##   However, this ignores `REML=TRUE`. Log-likelihood value is probably
##   inaccurate.
## Warning: Log-likelihood is corrected for models with transformed response.
##   However, this ignores `REML=TRUE`. Log-likelihood value is probably
##   inaccurate.
```

```
## # Indices of model performance
##
## AIC     |    AICc |     BIC | R2 (cond.) | R2 (marg.) |   ICC |  RMSE | Sigma
## ----------------------------------------------------------------------------
## 37989.3 | 37989.4 | 38058.5 |      0.843 |      0.287 | 0.780 | 0.365 | 0.390
```

```
# R2 (comparable metric)
r2(fit_time)
```

```
## # R2 for Mixed Models
##
##   Conditional R2: 0.843
##      Marginal R2: 0.287
```

5.2 Coding Time Model Inference

Calculates response-scale estimated marginal means and Tukey-adjusted contrasts for the time-model interaction.

```
# Estimate interaction-level coding times and multiplicity-adjusted contrasts.
emm_time <- emmeans(fit_time, ~ ai * group, type = "response")
```

```
## Note: D.f. calculations have been disabled because the number of observations exceeds 3000.
## To enable adjustments, add the argument 'pbkrtest.limit = 4000' (or larger)
## [or, globally, 'set emm_options(pbkrtest.limit = 4000)' or larger];
## but be warned that this may result in large computation time and memory use.
```

```
## Note: D.f. calculations have been disabled because the number of observations exceeds 3000.
## To enable adjustments, add the argument 'lmerTest.limit = 4000' (or larger)
## [or, globally, 'set emm_options(lmerTest.limit = 4000)' or larger];
## but be warned that this may result in large computation time and memory use.
```

```
pairs(emm_time, adjust = "tukey")
```

```
##  contrast                            ratio     SE  df null z.ratio p.value
##  FALSE clinicians / TRUE clinicians 1.621 0.1100 Inf    1   7.143 <0.0001
##  FALSE clinicians / FALSE coders    1.088 0.1500 Inf    1   0.612  0.9902
##  FALSE clinicians / TRUE coders     1.529 0.2330 Inf    1   2.786  0.0597
##  FALSE clinicians / FALSE students  1.497 0.1850 Inf    1   3.267  0.0139
##  FALSE clinicians / TRUE students   2.522 0.3510 Inf    1   6.642 <0.0001
##  TRUE clinicians / FALSE coders     0.672 0.1020 Inf    1  -2.613  0.0940
##  TRUE clinicians / TRUE coders      0.943 0.1300 Inf    1  -0.422  0.9983
##  TRUE clinicians / FALSE students   0.923 0.1290 Inf    1  -0.572  0.9929
##  TRUE clinicians / TRUE students    1.556 0.1920 Inf    1   3.582  0.0046
##  FALSE coders / TRUE coders         1.405 0.0984 Inf    1   4.851 <0.0001
##  FALSE coders / FALSE students      1.375 0.1900 Inf    1   2.305  0.1919
##  FALSE coders / TRUE students       2.317 0.3530 Inf    1   5.515 <0.0001
##  TRUE coders / FALSE students       0.979 0.1490 Inf    1  -0.139  1.0000
##  TRUE coders / TRUE students        1.650 0.2280 Inf    1   3.622  0.0040
##  FALSE students / TRUE students     1.685 0.1140 Inf    1   7.728 <0.0001
##
## Degrees-of-freedom method: asymptotic
## P value adjustment: tukey method for comparing a family of 6 estimates
## Tests are performed on the log scale
```

```
confint(pairs(emm_time))
```

```
##  contrast                            ratio     SE  df asymp.LCL asymp.UCL
##  FALSE clinicians / TRUE clinicians 1.621 0.1100 Inf     1.337      1.97
##  FALSE clinicians / FALSE coders    1.088 0.1500 Inf     0.734      1.61
##  FALSE clinicians / TRUE coders     1.529 0.2330 Inf     0.990      2.36
##  FALSE clinicians / FALSE students  1.497 0.1850 Inf     1.053      2.13
##  FALSE clinicians / TRUE students   2.522 0.3510 Inf     1.696      3.75
##  TRUE clinicians / FALSE coders     0.672 0.1020 Inf     0.435      1.04
##  TRUE clinicians / TRUE coders      0.943 0.1300 Inf     0.636      1.40
##  TRUE clinicians / FALSE students   0.923 0.1290 Inf     0.621      1.37
```

```
##  TRUE clinicians / TRUE students    1.556 0.1920 Inf     1.095      2.21
##  FALSE coders / TRUE coders         1.405 0.0984 Inf     1.151      1.72
##  FALSE coders / FALSE students      1.375 0.1900 Inf     0.927      2.04
##  FALSE coders / TRUE students       2.317 0.3530 Inf     1.501      3.58
##  TRUE coders / FALSE students       0.979 0.1490 Inf     0.634      1.51
##  TRUE coders / TRUE students        1.650 0.2280 Inf     1.113      2.45
##  FALSE students / TRUE students     1.685 0.1140 Inf     1.390      2.04
##
## Degrees-of-freedom method: asymptotic
## Confidence level used: 0.95
## Conf-level adjustment: tukey method for comparing a family of 6 estimates
## Intervals are back-transformed from the log scale
```

Prints the response-scale estimated marginal means from the coding-time model.

```
# Display the interaction-level time estimates used for interpretation.
summary(emm_time)
```

```
##  ai    group      response   SE  df asymp.LCL asymp.UCL
##  FALSE clinicians     85.8 8.45 Inf      70.8     104.1
##   TRUE clinicians     53.0 5.22 Inf      43.7      64.2
##  FALSE coders         78.9 9.18 Inf      62.8      99.1
##   TRUE coders         56.2 6.53 Inf      44.7      70.5
##  FALSE students       57.4 5.65 Inf      47.3      69.6
##   TRUE students       34.0 3.35 Inf      28.1      41.3
##
## Degrees-of-freedom method: asymptotic
## Confidence level used: 0.95
## Intervals are back-transformed from the log scale
```

Calculates response-scale marginal means and contrasts for AI use in the coding-time model.

```
# Test the average AI-use contrast for coding time.
emm_time_ai <- emmeans(fit_time, ~ ai, type = "response")
```

```
## Note: D.f. calculations have been disabled because the number of observations exceeds 3000.
## To enable adjustments, add the argument 'pbkrtest.limit = 4000' (or larger)
## [or, globally, 'set emm_options(pbkrtest.limit = 4000)' or larger];
## but be warned that this may result in large computation time and memory use.
```

```
## Note: D.f. calculations have been disabled because the number of observations exceeds 3000.
## To enable adjustments, add the argument 'lmerTest.limit = 4000' (or larger)
## [or, globally, 'set emm_options(lmerTest.limit = 4000)' or larger];
## but be warned that this may result in large computation time and memory use.
```

```
## NOTE: Results may be misleading due to involvement in interactions
```

```
summary(emm_time_ai)
```

```
##  ai    response   SE  df asymp.LCL asymp.UCL
##  FALSE     73.0 5.18 Inf      63.5      83.8
##   TRUE     46.6 3.31 Inf      40.6      53.6
##
## Results are averaged over the levels of: group
## Degrees-of-freedom method: asymptotic
## Confidence level used: 0.95
## Intervals are back-transformed from the log scale
```

```r
pairs(emm_time_ai)
```

```
##  contrast     ratio    SE  df null z.ratio p.value
##  FALSE / TRUE  1.57 0.103 Inf    1   6.828 <0.0001
##
## Results are averaged over the levels of: group
## Degrees-of-freedom method: asymptotic
## Tests are performed on the log scale
```

```r
confint(pairs(emm_time_ai))
```

```
##  contrast     ratio    SE  df asymp.LCL asymp.UCL
##  FALSE / TRUE  1.57 0.103 Inf      1.38      1.78
##
## Results are averaged over the levels of: group
## Degrees-of-freedom method: asymptotic
## Confidence level used: 0.95
## Intervals are back-transformed from the log scale
```

5.3 Coding-Time Model Visualization

Plots and saves predicted coding time and confidence intervals by group and AI use.

```r
# Convert response-scale estimated marginal means to a plotting data frame.
# lmer confidence limits are retained under standardized names for consistency
# with the Jaccard prediction data used in the combined figure.
# Request response-scale estimates and confidence intervals explicitly.
plot_time <- as.data.frame(
  confint(emm_time, type = "response")
)

# Identify the confidence-limit names returned by the installed emmeans
# version and the degrees-of-freedom method used for this model.
lower_ci_col <- intersect(
  c("lower.CL", "asymp.LCL"),
  names(plot_time)
)

upper_ci_col <- intersect(
  c("upper.CL", "asymp.UCL"),
  names(plot_time)
)

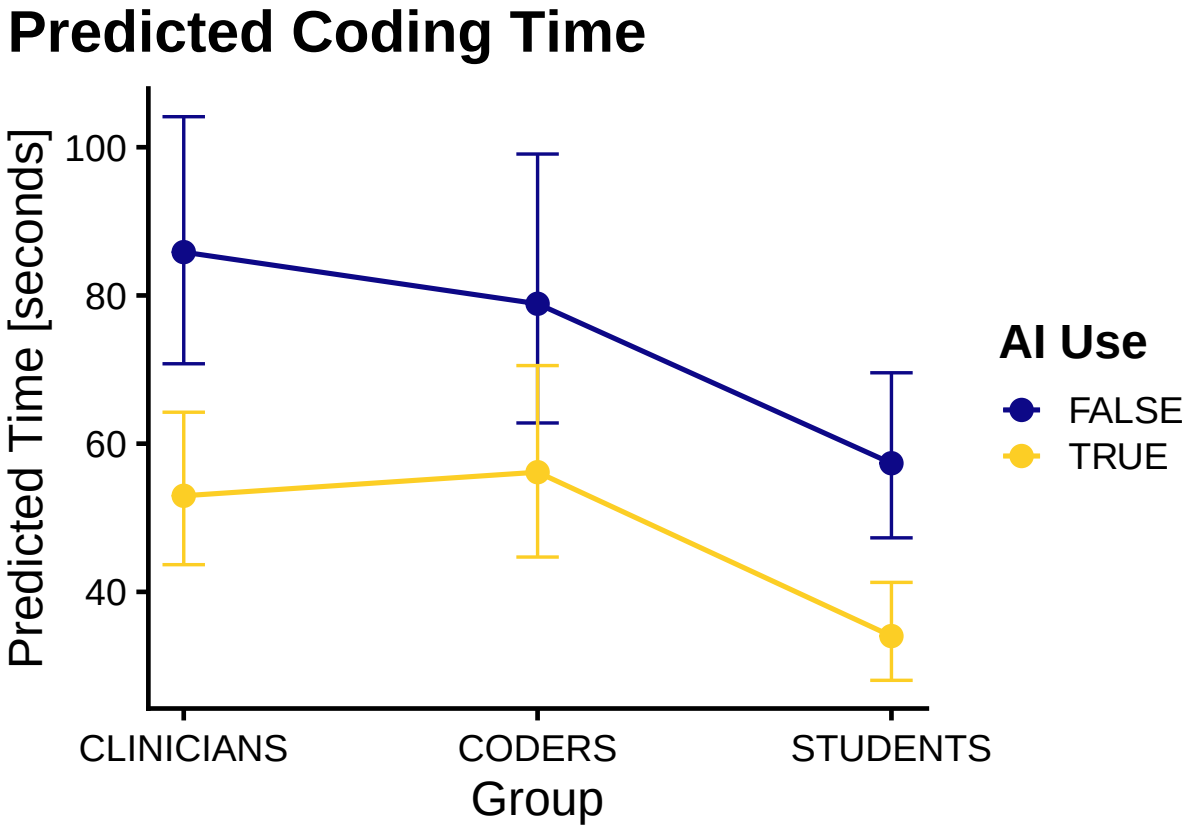
# Stop with an informative error if confidence limits are unavailable.
if (length(lower_ci_col) == 0 || length(upper_ci_col) == 0) {
  stop(
    "Confidence-limit columns were not found. Available columns are: ",
    paste(names(plot_time), collapse = ", ")
  )
}

# Standardize the confidence-limit names for downstream plotting.
plot_time <- plot_time %>%
  transmute(
    ai,
```

```
    group = toupper(group),
    response,
    lower.CL = .data[[lower_ci_col[1]]],
    upper.CL = .data[[upper_ci_col[1]]]
  )

p_time_emmeans <- ggplot(
  plot_time,
  aes(x = group, y = response, color = ai, group = ai)
) +
  # CI bars first so lines/points sit on top
  geom_errorbar(
    aes(ymin = lower.CL, ymax = upper.CL),
    width = 0.12,
    linewidth = 0.6
  ) +
  geom_line(
    linewidth = 0.9
  ) +
  geom_point(
    size = 3.2
  ) +
  # Optional: keep within Jaccard bounds (0-1) without chopping CIs off harshly
  # coord_cartesian(ylim = c(0.4, 0.8), expand = FALSE) +
  scale_x_discrete(expand = expansion(add = 0.1)) +  # slight buffer (not 0 to avoid clipping)
  # coord_cartesian(ylim = c(0.4, 0.8), clip = "off") +
  theme(plot.margin = margin(10, 20, 10, 10)) +
  # scale_y_continuous(
  #   # breaks = seq(0, 1, 0.1),
  #   labels = function(x) sprintf("%.1f", x)
  # ) +
  scale_color_viridis_d(end = 0.9, option = "C") +
  labs(
    title = "Predicted Coding Time",
    x = "Group",
    y = "Predicted Time [seconds]",
    color = "AI Use"
  ) +
  theme_classic(base_size = 18) +
  theme(
    plot.title.position = "plot",
    plot.title = element_text(face = "bold", size = 22),
    plot.subtitle = element_text(size = 16, margin = margin(b = 10)),
    axis.title = element_text(size = 18),
    axis.text  = element_text(size = 14, color = "black"),
    legend.position = "right",
    legend.title = element_text(face = "bold"),
    legend.text = element_text(size = 14),
    panel.grid = element_blank(),
    axis.text.x = element_text(margin = margin(t = 5))
  )

p_time_emmeans
```




```
ggsave(
  "../plots/time_emmeans.png",
  plot = p_time_emmeans,
  width = 8,
  height = 6,
  dpi = 300
)
```

5. Misc Plots for Poster

Recreates the Jaccard and time prediction panels and saves them as a combined figure with a shared legend.

```
# Assemble publication-style model panels
# --- Plot 1: Jaccard ---
p_beta <- ggplot(
  plot_beta,
  aes(x = group, y = response, color = ai, group = ai)
) +
  geom_errorbar(
    aes(ymin = lower.CL, ymax = upper.CL),
    width = 0.12, linewidth = 0.6
  ) +
  geom_line(linewidth = 0.9) +
  geom_point(size = 3.2) +
  scale_x_discrete(expand = expansion(add = 0.15)) +
  coord_cartesian(ylim = c(0.4, 0.8), clip = "off") +
  scale_y_continuous(
    breaks = seq(0, 1, 0.1),
```

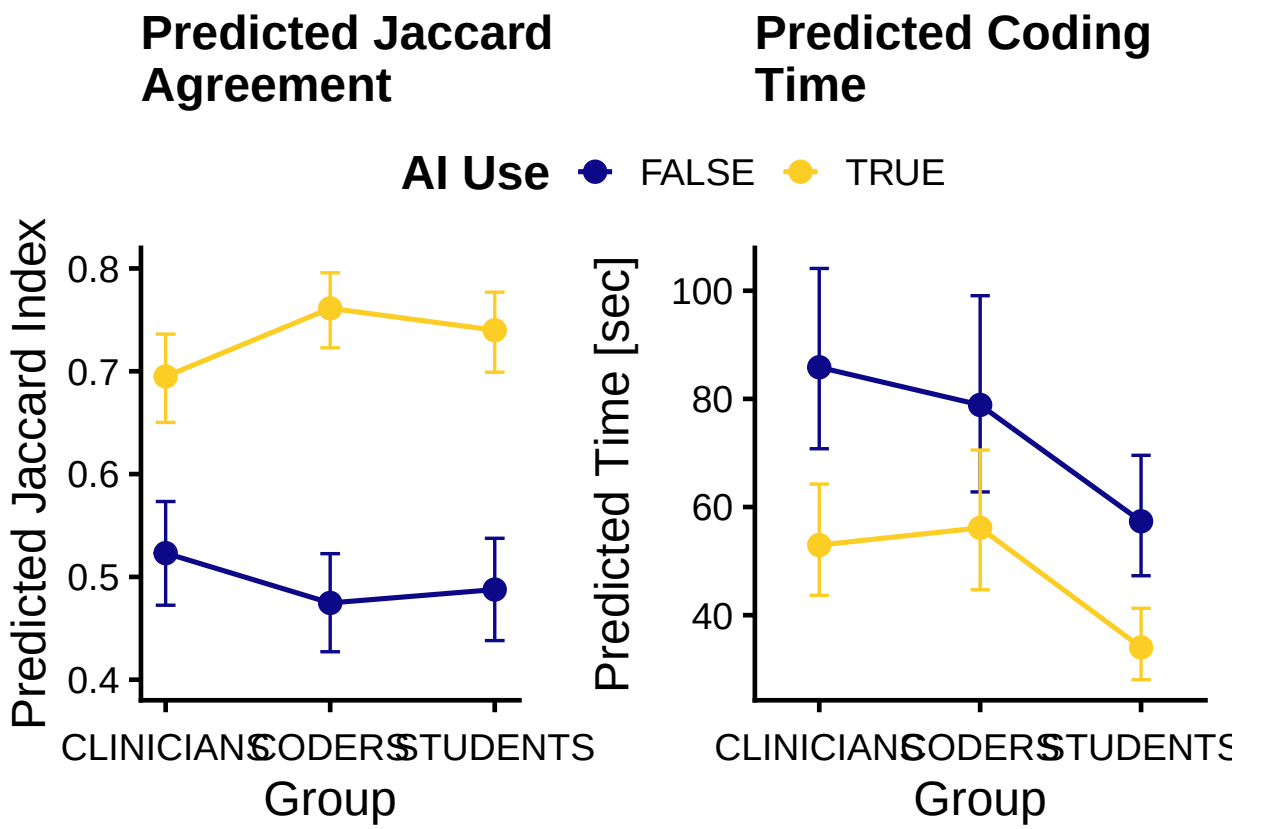
```
    labels = function(x) sprintf("%.1f", x)
  ) +
  scale_color_viridis_d(end = 0.9, option = "C") +
  labs(
    title = "Predicted Jaccard\nAgreement",
    x = "Group",
    y = "Predicted Jaccard Index",
    color = "AI Use"
  ) +
  theme_classic(base_size = 18) +
  theme(
    plot.margin = margin(10, 5, 12, 10),  # more bottom space
    plot.title = element_text(face = "bold", size = 18, margin = margin(b = 5)),
    axis.title = element_text(size = 18),
    axis.text  = element_text(size = 14, color = "black"),
    axis.text.x = element_text(margin = margin(t = 8)), # separate labels
    legend.title = element_text(face = "bold"),
    legend.text = element_text(size = 14)
  )

# --- Plot 2: Time ---
p_time <- ggplot(
  plot_time,
  aes(x = group, y = response, color = ai, group = ai)
) +
  geom_errorbar(
    aes(ymin = lower.CL, ymax = upper.CL),
    width = 0.12, linewidth = 0.6
  ) +
  geom_line(linewidth = 0.9) +
  geom_point(size = 3.2) +
  scale_x_discrete(expand = expansion(add = 0.4)) +
  scale_color_viridis_d(end = 0.9, option = "C") +
  labs(
    title = "Predicted Coding\nTime",
    x = "Group",
    y = "Predicted Time [sec]",   #  restored
    color = "AI Use"
  ) +
  theme_classic(base_size = 18) +
  theme(
    plot.margin = margin(10, 40, 12, 5),  #  BIG right margin (fix clipping)
    plot.title = element_text(face = "bold", size = 18, margin = margin(b = 5)),
    axis.title = element_text(size = 18),
    axis.title.y = element_text(margin = margin(r = 12)),  #  push label outward
    axis.text  = element_text(size = 14, color = "black"),
    axis.text.x = element_text(margin = margin(t = 8)),   #  prevent overlap
    legend.title = element_text(face = "bold"),
    legend.text = element_text(size = 14)
  )

# --- Combine with spacer (KEY FIX) ---
combined <- (p_beta | plot_spacer() | p_time) +
```

```
  plot_layout(
    widths = c(1.05, 0.06, 1.25),    #  small gap between panels
    guides = "collect"
  ) &
  theme(
    legend.position = "top",
    legend.direction = "horizontal",
    legend.box = "horizontal",
    legend.key.width = unit(1.1, "lines"),
    legend.spacing.x = unit(0.4, "lines"),
    plot.margin = margin(5, 5, 5, 5)
  )

combined
```




```
ggsave(
  "../plots/combined.png",
  plot = combined,
  # <-- increase this
  width = 8,
  height = 6,
  dpi = 300
)
```

Builds and saves a bar chart of manually entered GatorTron performance metrics and confidence intervals.

```
# Visualize externally supplied model-performance values; these values are hard-coded rather than calcu
gatortron_metrics_df <- tibble::tribble(
```

```
  ~Metric,          ~Estimate, ~CI_half,
  "F1",               76.9,      0.14,
  "Precision",        81.5,      1.74,
  "Recall",           74.4,      1.14,
  "Exact Match",      52.2,      0.66
) %>%
  mutate(
    CI_low  = Estimate - CI_half,
    CI_high = Estimate + CI_half,
    Metric  = factor(Metric, levels = Metric)
  )

p_bar <- ggplot(gatortron_metrics_df, aes(x = Metric, y = Estimate, fill = Metric)) +
  geom_col(width = 0.65, color = "white", linewidth = 0.4) +

  geom_errorbar(
    aes(ymin = CI_low, ymax = CI_high),
    width = 0.12,
    linewidth = 0.9
  ) +

  geom_text(
    aes(label = sprintf("%.1f%%", Estimate)),
    vjust = -2.75,
    size = 5,
    color = "black",
    fontface = "bold"
  ) +

  geom_text(
    aes(label = sprintf("[%.1f, %.1f]", CI_low, CI_high)),
    vjust = -2.0, size = 4
  ) +

  scale_y_continuous(
    breaks = seq(50, 100, 10),
    labels = function(x) sprintf("%.0f%%", x)
  ) +

  coord_cartesian(ylim = c(50, 100), expand = FALSE) +

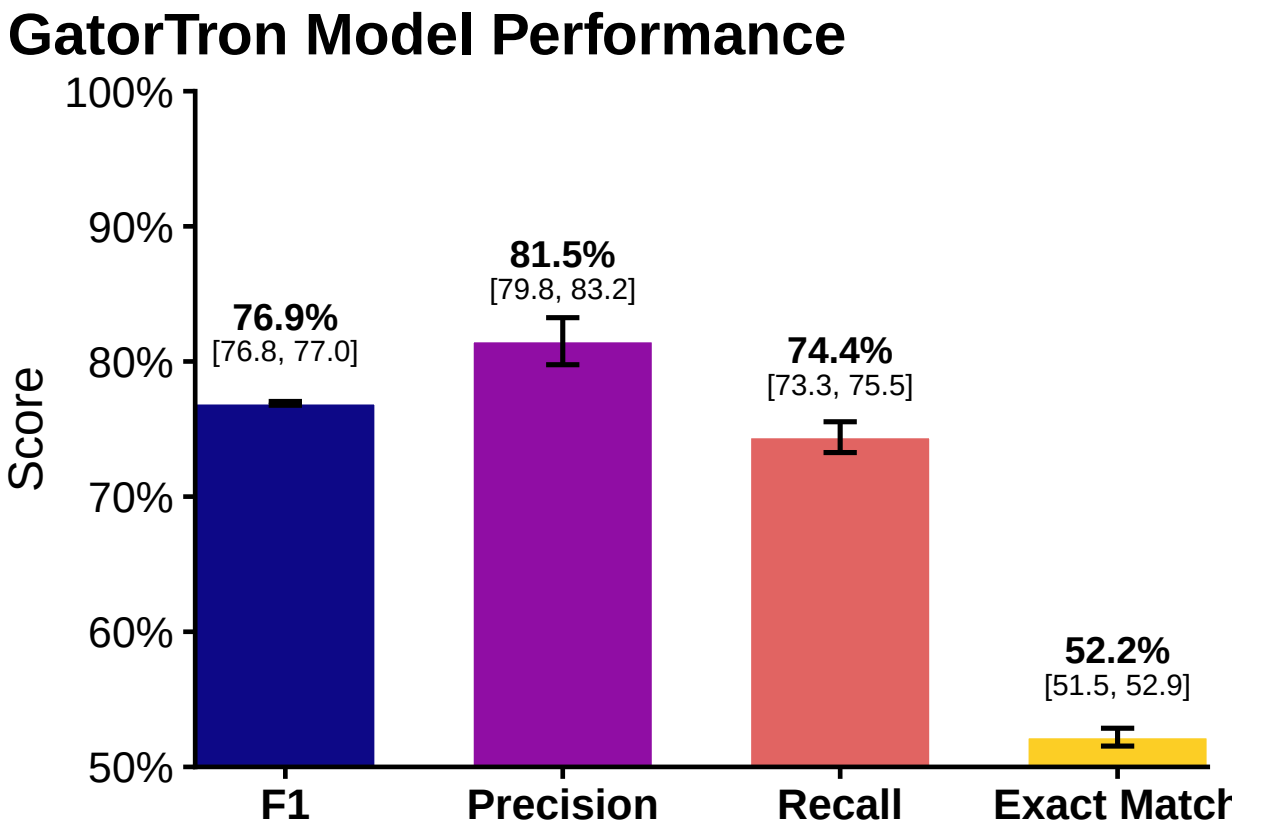
  scale_fill_viridis_d(end = 0.9, option = "C", guide = "none") +

  labs(
    title = "GatorTron Model Performance",
    x = NULL,
    y = "Score"
  ) +

  theme_classic(base_size = 18) +
  theme(
    plot.title.position = "plot",
    plot.title = element_text(face = "bold", size = 22),
```

```
    plot.subtitle = element_text(size = 16, margin = margin(b = 10)),
    axis.title = element_text(size = 18),
    axis.text  = element_text(size = 16, color = "black"),
    axis.text.x = element_text(face = "bold"),
    panel.grid = element_blank()
  )

p_bar
```




```
ggsave(
  "../plots/metrics.png",
  plot = p_bar,
  # <-- increase this
  dpi = 300
)
```

```
## Saving 6.5 x 4.5 in image
```